LeafLens

AI-Powered Leaf Disease Prediction System

Low-Level Design (LLD)

Prepared by

Abhijeet Kumar

DA24B001

1 Introduction

The Low-Level Design describes how the individual parts of LeafLens are implemented and how they interact during execution.

The High-Level Design explains why the application is divided into separate components. This document takes the next step and describes what happens inside those components.

The main focus is on the FastAPI backend, the model inference service, image validation, API communication, monitoring instrumentation, and the tests used to verify the implementation.

1.1 Scope

This document covers:

- Backend module organisation.
- Responsibilities of the main implementation files.
- REST API endpoint specifications.
- Image upload and validation logic.
- Model inference sequence.
- Error handling and monitoring.
- Unit and integration testing.

2 Backend Module Design

The backend is organised into smaller modules rather than keeping all application logic inside a single file.

This makes the code easier to understand and allows different responsibilities to be tested independently.

Table 1: Important backend modules

File / Module	Responsibility
`backend/app/main.py`	Initialises FastAPI, manages model loading, handles API requests, validates uploads, and exposes endpoints.
`backend/app/core/metrics.py`	Defines Prometheus counters and histograms used to observe API requests and prediction behaviour.
`backend/app/schemas/prediction.py`	Defines Pydantic response models used to structure prediction results.
`ml/serving/predictor.py`	Loads the trained model checkpoint, preprocesses input images, and performs model inference.
`backend/tests/test_prediction_api.py`	Contains automated tests for API behaviour, validation, prediction, and monitoring metrics.

2.1 Application Initialisation

When the FastAPI application starts, its lifespan logic loads the trained model through the prediction component.

Loading the model during application initialisation avoids repeatedly loading the checkpoint for every incoming prediction request.

The backend then makes the prediction service available to the request-handling functions.

3 API Design

The backend exposes a REST API that acts as the communication boundary between the frontend and the machine learning component.

The API currently provides four main endpoints.

Table 2: API endpoint summary

Method	Endpoint	Input	Purpose
GET	`/`	None	Returns the basic application response.
GET	`/health`	None	Checks backend health and model readiness.
POST	`/predict`	Image file	Accepts an image and returns the classification result.
GET	`/metrics`	None	Exposes Prometheus metrics for monitoring.

3.1 Root Endpoint: GET /

Purpose: Provides a basic endpoint to confirm that the application is responding.

Input: No request body is required.

Output: A basic application response.

3.2 Health Endpoint: GET /health

Purpose: Provides a health-check endpoint for checking backend availability.

Docker Compose uses this endpoint to determine whether the backend is ready.

Input: No request body is required.

Output: A health status response. The application also exposes its model version in the health response when available.

3.3 Prediction Endpoint: POST /predict

This is the main application endpoint. It accepts an uploaded image, validates it, and passes it to the prediction service.

Request type: Multipart file upload.

Input: A leaf image submitted as an uploaded file.

Processing:

1. Read the uploaded file.
2. Check whether the file is empty.
3. Enforce the 10 MB upload-size limit.
4. Attempt to decode the image.
5. Convert the image to RGB.
6. Call the prediction component.
7. Return the structured prediction response.

Output: A JSON response structured using the project's Pydantic prediction schema.

The response fields are defined by `PredictionItem` and `PredictionResponse` in the prediction schema module.

3.4 Metrics Endpoint: GET /metrics

Purpose: Exposes the backend's Prometheus instrumentation.

Input: No request body is required.

Output: Metrics in Prometheus exposition format, including request, duration, and prediction statistics.

4 Image Validation and Preprocessing

Image validation is performed before inference. This is important because an invalid upload should not be passed to the model.

The backend currently enforces a maximum upload size of 10 MB.

Table 3: Input validation rules

Condition	Handling	Reason
Empty upload	Reject request	There is no image content to process.
Oversized file	Reject request	Prevents uploads beyond the configured limit.
Invalid image	Reject request	The content cannot be decoded as an image.
Non-RGB image	Convert to RGB	Provides a consistent colour-channel format.
Valid image	Continue to inference	The image is ready for model preprocessing.

4.1 Inference Preprocessing

After backend validation, the prediction component applies the transformations required by the trained model.

The current inference pipeline includes:

- Resize the image to 224×224 pixels.
- Convert the image into a PyTorch tensor.
- Apply ImageNet mean and standard deviation normalisation.
- Pass the resulting tensor to the model.

The preprocessing steps are kept consistent with the model's training configuration.

5 Prediction Processing Sequence

The following flowchart describes what happens when a user submits an image.

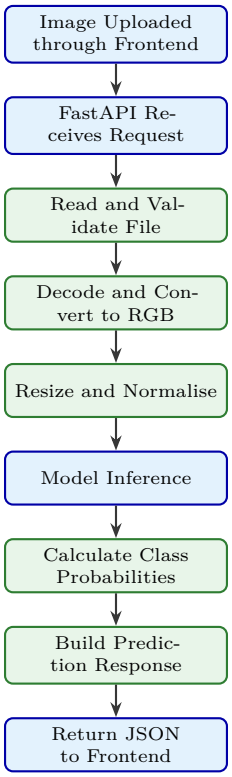


Figure 1: Detailed prediction sequence

5.1 Model Inference

The predictor loads the saved MobileNetV3-Large checkpoint.

The device selection logic uses Apple Metal Performance Shaders (MPS) when available and falls back to CPU otherwise.

During inference, the model produces class scores. Softmax is applied to obtain class probabilities, which are used to determine the predicted class and its confidence.

The inference component is kept separate from the API handler so that model-specific operations do not become mixed with HTTP request processing.

6 Error Handling

The application handles expected input errors before inference and records unexpected prediction failures.

Table 4: Error handling behaviour

Situation	Response	Handling
Empty upload	Client error	Reject the request before image processing.
Oversized upload	Client error	Reject the file when it exceeds 10 MB.
Invalid image	Client error	Return a validation error.
Prediction exception	HTTP 500	Log the failure, increment the error metric, and return a generic error response.

The backend avoids exposing internal exception details to the user. Detailed information is recorded through application logging for debugging.

7 Monitoring Instrumentation

Monitoring is implemented directly in the backend using the Prometheus client library.

The instrumentation records both general API activity and prediction-specific behaviour.

Table 5: Backend monitoring metrics

Metric	Purpose
HTTP request counter	Counts requests grouped by method, endpoint, and status.
HTTP request duration histogram	Records the time taken to process requests.
Prediction counter	Counts successful predictions grouped by class.
Prediction confidence histogram	Records the distribution of prediction confidence.
Prediction error counter	Counts failures during prediction processing.

A middleware component records request counts and durations. Prediction-specific metrics are updated when the prediction operation succeeds or fails.

Prometheus collects the metrics, and Grafana displays them in the LeafLens monitoring dashboard.

8 Testing Design

Testing is used to verify that the API behaves as expected under both valid and invalid conditions.

The backend test suite contains eight tests.

Table 6: Backend test cases

ID	Test case	Expected behaviour
T01	Root endpoint	The root endpoint responds successfully.
T02	Health endpoint	The health endpoint returns the expected status.
T03	Successful prediction	A valid image produces a prediction response.
T04	Empty upload	The API rejects an empty file.
T05	Invalid image	The API rejects unreadable image content.
T06	Oversized upload	The API rejects a file exceeding the size limit.
T07	Metrics endpoint	Prometheus metrics are exposed successfully.
T08	Prediction metrics	A successful prediction updates the relevant metrics.

All eight backend tests passed during the recorded test run.

8.1 Testing Approach

The tests use FastAPI's testing support to exercise the API without requiring a manually operated browser.

They cover the normal prediction path, validation failures, and monitoring instrumentation.

This gives us confidence that the backend handles the expected request types and that its monitoring metrics are updated as intended.

9 Deployment Configuration

The application is deployed using Docker Compose with four independent services.

Table 7: Service configuration

Service	Container Port	Host Port
FastAPI backend	8000	8001
Streamlit frontend	8501	8501
Prometheus	9090	9092
Grafana	3000	3002

The backend health check is used by Docker Compose to verify readiness. The frontend depends on the backend becoming healthy before it starts.

Prometheus scrapes the backend's metrics endpoint, and Grafana reads the collected data through the configured Prometheus data source.

10 Conclusion

The Low-Level Design shows how the main LeafLens components work at implementation level.

The backend is responsible for receiving and validating requests, while the inference component handles image transformations and classification. Prometheus instrumentation records the application's behaviour, and the automated tests verify the important API paths.

Keeping these responsibilities separate makes the system easier to debug, test, and extend.

The design also gives a clear connection between the architecture described in the earlier document and the code that runs the application.